Artificial Intelligence
CS-202

Topic: Game Playing and Adversarial Search

Habiba Arshad

Department of Computer Science

University of Wah

Week-5 and 6 CLO

This week lecture will cover the following CLO's:

- **CLO1:** Discuss the key concepts in the field of artificial intelligence
- **CLO 2: Implement** artificial intelligence techniques and case studies

CLO's Mapping

CLO 1:

- Introduction
  - Game Playing in AI
  - Adversarial Search and the Minimax Algorithm
  - Cut-Off Search
  - Pruning Techniques (including Alpha-Beta Pruning)

CLO 2:

- Implement
  - Adversarial Search and the Minimax Algorithm
  - Alpha-Beta Pruning
  - Case Studies: Chess and Checkers

Single Agent environment

- Until now, we have discussed single agent environment
  - only one person or agent searching the solution space to find the goal or the solution.
  - often expressed in the form of a sequence of actions

- There might be some situations where more than one agent/person is searching for the solution in the same search space
  - usually occurs in game playing where two opponents (adversaries) are searching for a goal.
  - Multi-agent environment

Multi-agent environment

- The environment with more than one agent
- Each agent is an opponent of other agent and playing against each other
- Each agent needs to consider the action of other agent and effect of that action on their performance.

Introduction to Game Playing in AI

- **Definition:** Game playing is a fundamental problem area for AI that focuses on creating intelligent agents to compete in games like chess, checkers, and more..
- **Types of Games:** Deterministic (e.g., Chess, Checkers)Stochastic (e.g., Backgammon)The agent operates within a state space, where each state represents a configuration of the world.

Key Concepts:

- **Adversarial Search:** Search strategies for games where agents compete against each other.
- **Goal:** Develop strategies to maximize an agent's winning chances.

What is Adversarial Search

Definition

- A type of search used in environments where agents (players) compete against each other. It is primarily used in games like Chess, Go, and Checkers, where each player's goal is to maximize their own payoff while minimizing that of their opponent.

Examples of Adversarial Games

- Turn-Based Games: Chess, Checkers, Tic-Tac-Toe
- Simultaneous Games: Poker, Rock-Paper-Scissors

Contrast with Regular Search

- Unlike typical search problems, adversarial search involves dealing with uncertainty and the strategic counteractions of opponents.

Adversarial Search Overview

- The Adversarial search is a well-suited approach in a competitive environment, where two or more agents have conflicting goals. The adversarial search can be employed in two-player zero-sum(utility/ payoff) based games which means what is good for one player will be the misfortune for the other. In such a case, there is no win-win outcome.
- Adversarial search algorithms are the backbone of strategic decision-making in artificial intelligence, it enables the agents to navigate competitive scenarios effectively.
- By employing adversarial search, AI agents can make optimal decisions while anticipating the actions of an opponent with their opposing objectives.

Why Adversarial Search is Important in AI

Strategic Decision-Making

- Adversarial search equips AI agents with strategies to anticipate and counteract the moves of opponents.

Optimal Play

- Helps in formulating strategies that can lead to the best possible outcomes, even in worst-case scenarios.

Application Beyond Games

- The principles are applicable in fields like cybersecurity, economic modeling, and automated negotiations.

Modelling Human-Like Behaviour

- By playing games and adapting strategies, AI systems can simulate human-like strategic reasoning.

Types of Games in AI

Perfect information

A game in which agents can look into the complete board.

Agents have all the information about the game, and they can see each other moves also.

Examples are Chess, tic-tac-toe, Checkers, Go, etc.

Imperfect information

A game in which agents do not have all information about the game and not aware with what's going on,

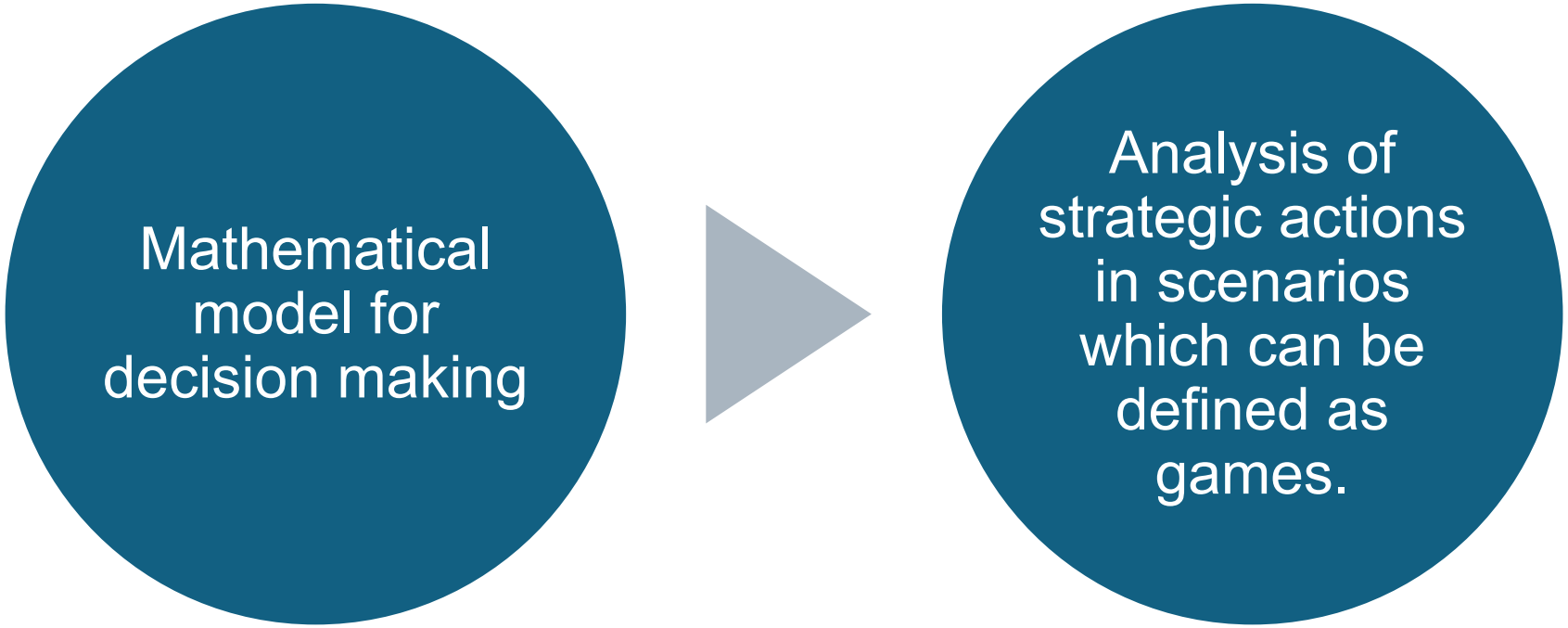
Examples are Battleship, blind, Bridge, etc.

Contd..

- **Deterministic games**
  - Those games which **follow a strict pattern and set of rules** for the games, and there is **no randomness** associated with them.
  - Examples are chess, Checkers, Go, tic-tac-toe, etc.

- **Non-deterministic games/ Stochastic games**
  - Those games which **have various unpredictable events** and has a **factor of chance or luck.**
  - This factor of chance or luck is introduced by either dice or cards. These are random, and each action response is not fixed.
  - Also called as stochastic games.
  - Example: Backgammon, Monopoly, Poker, etc.

Game Theory


Mathematical model for decision making
Analysis of strategic actions in scenarios which can be defined as games.

Grades Game

- Two players secretly and simultaneously make a choice, with a defined outcome
  - Choices: Choose alpha (α) or beta (β)
- You choose α they choose α: B-
- You choose α they choose β: A
- You choose β they choose α : C
- You choose β they choose β: B+

Grades Game

		Partner	
		α	β
Me	α	B-	A
	β	C	B+

My Grades

Game1: Grade Game

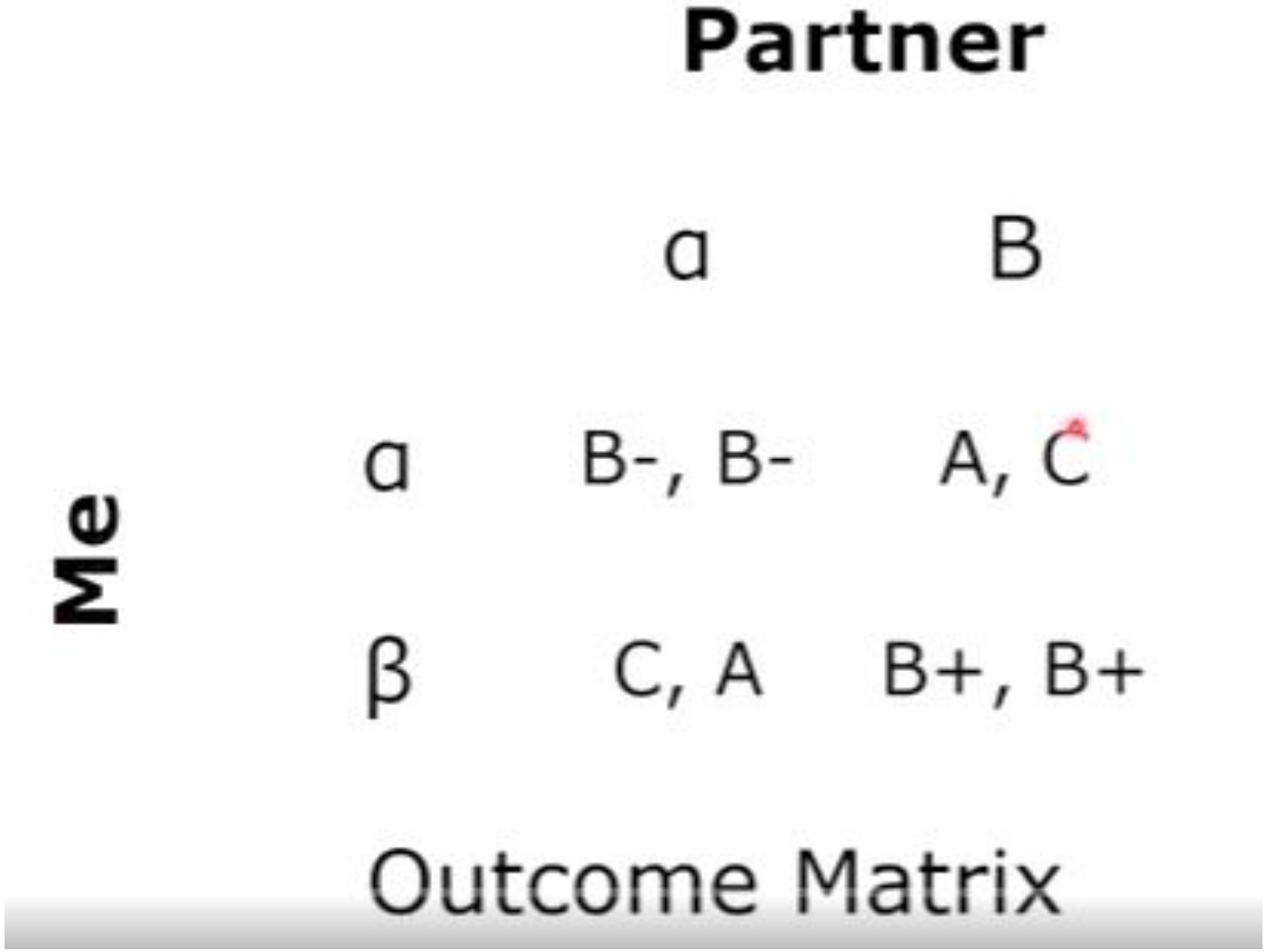
Me \ Partner	α	β
α	B-	A
β	C	B+

My Grades

Me \ Partner	α	β
α	B-	C
β	A	B+

Partner Grades

Grades Game



Grades Game

- We have actions, strategies, outcomes
- Not yet a game, what are we missing?

Grade Game

- We have actions, strategies, outcomes
- Not yet a game, what are we missing?
- Payoffs **(Utility)**
  - What is our objective or goal in the game?
  - Game theory can't help decide your objectives / payoffs
  - If you have payoffs, GT can help you achieve them
- Examples:
  - We care about our own grade
  - We care about others' grades

Payoff (Utility) Example

- Payoff = assign numerical value to outcome
  - How much do I like the outcome?
  - The more I desire it, the higher the payoff
- Example Payoff
  - Imagine one of the players is greedy and assigns higher payoffs if they do better than partner
  - If both get B-, payoff is neural (0)
  - If I get A and you get C, payoff is good (3)

Grades Game Payoffs

Me \ Partner	α	B
α	B-, B-	A, C
β	C, A	B+, B+

Grade Results

Me \ Partner	α	β
α	0, 0	3, -1
β	-1, 3	1, 1

Payoff Matrix

Grades Game Payoffs

Me \ Partner	α	β
α	0, 0	3, -1
β	-1, 3	1, 1

Payoff Matrix

- Numbers = Utility
- Player should attempt to maximize utility
- This example
  - (B-,B-) -> (0)
  - (A, C) -> (3)
  - Player cares about self

Grades Game Payoffs

Partner

Me \ Partner	α	β
α	0, 0	3, -1
β	-1, 3	1, 1

Payoff Matrix

- What should you do?
- Consider all choices
- Partner choose α
  - You α = 0, β = -1
- Partner choose β
  - You α = 3, β = 1

- What is the best choice?

Grades Game Payoffs

Partner

Me \ Partner	α	β
α	0, 0	3, -1
β	-1, 3	1, 1

Payoff Matrix

- What should you do?
- Consider all choices
- Partner choose α
  - You α = 0, β = -1
- Partner choose β
  - You α = 3, β = 1
- α always best choice

Zero-Sum Game

- The games most commonly studied within AI (such as chess, tic-tac-toe and Go) are what game theorists call deterministic, two-player, turn-taking, perfect information, zero-sum games.
- "Perfect information" is a synonym for "fully observable,"1 and "zero-sum" means that what is good for one player is just as bad for the other: there is no "win-win" outcome.
- For games we often use the term move as a synonym for "action" and position as a synonym for "state."

Zero-Sum Game

- Two players MAX and MIN.
- MAX moves first, and then the players take turns moving until the game is over. At the end of the game, points are awarded to the winning player and penalties are given to the loser.
- One player of the game try to maximize one single value, while other player tries to minimize it.
- Each move by one player in the game is called as **ply**.

Formalization of the problem

- A game can be defined as a type of search in AI which can be formalized of the following elements:
- **Initial state (S_0)**
  - It specifies how the game is set up at the start.
- **Player(s) – To_Move(s)**
  - It specifies which player has moved in the state space.
- **Action(s)**
  - It returns the set of legal moves in state space.
- **Result(s, a)**
  - It is the transition model, which specifies the result of moves in the state space.

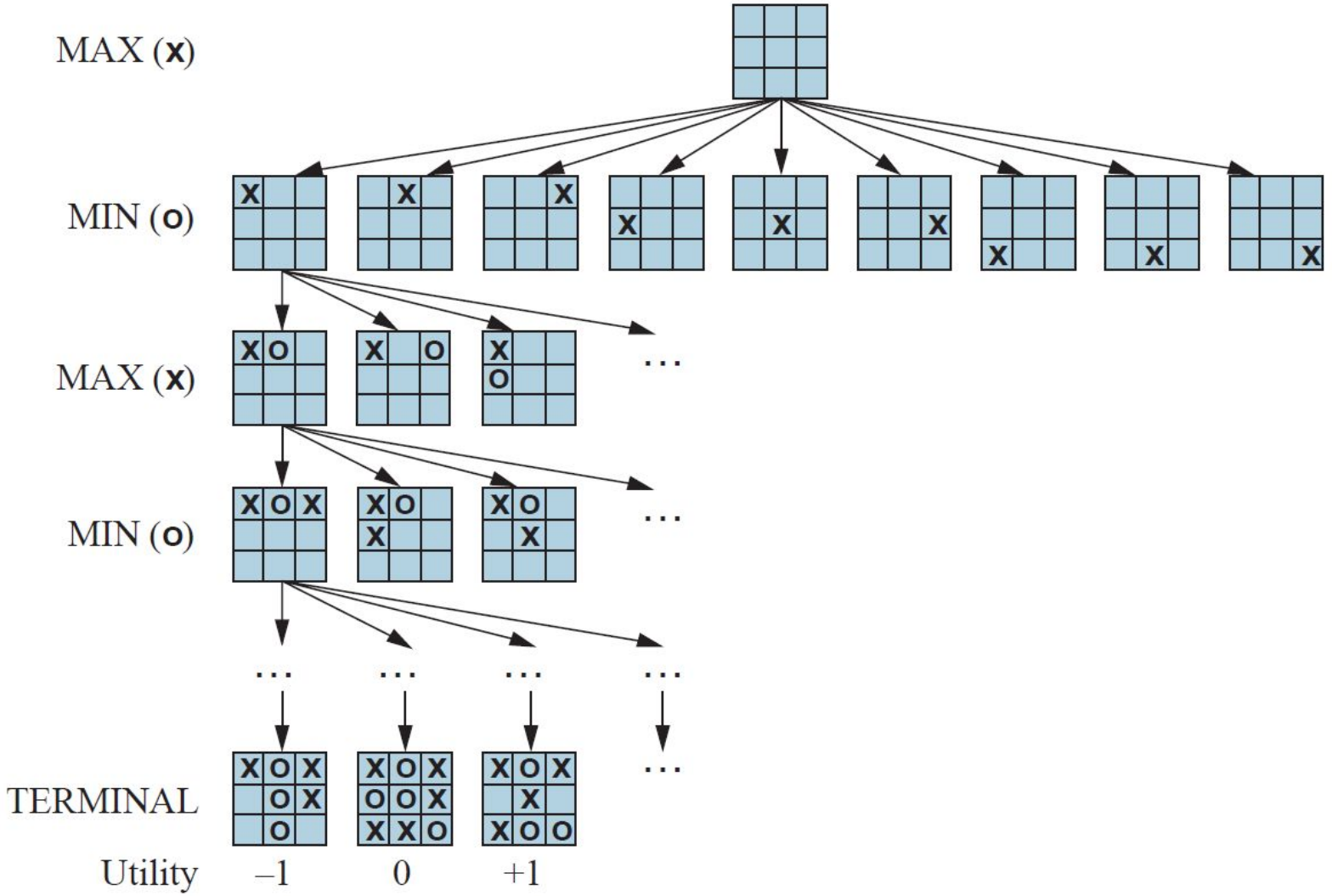
Contd..

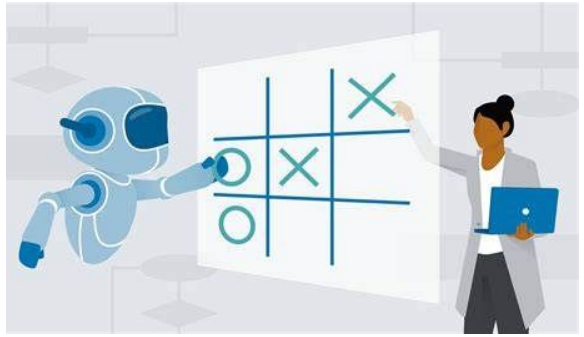
- **Terminal-Test(s)**
  - The state where the game ends is called terminal states.
  - Terminal test is true if the game is over, else it is false at any case.
- **Utility(s, p)**
  - A utility function gives the final numeric value for a game that ends in terminal states s for player p. It is also called payoff function. (**Reward after winning the game**)
  - For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, ½. And for tic-tac-toe, utility values are +1, -1, and 0

Game tree

- The initial state, Actions functions and Result function define game tree for a game.
- A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players.
- Also named as search tree and space graph.

Tic-Tac-Toe game tree





Mini-Max Algorithm

- A recursive or **backtracking** algorithm
  - The algorithm proceeds all the way down to the terminal node of the tree, compare the values and backtrack the tree to root node to decide the move.
- Performs a depth-first search algorithm for the exploration of the complete game tree.
- **Best Move strategy used**
  - Both players will adopt best move to not allow the opponent to win
- Computes the minimax decision for the current state
  - Max will try to maximize its utility (Best move)
  - Min will try to minimize the utility (Worst move)

Mini-Max Algorithm

- **Why not BFS?**
  BFS expands the game tree level-by-level and requires storing all nodes at each level, causing massive memory usage.
- Minimax needs to evaluate terminal states first, and recursion with backtracking is only possible using DFS. DFS goes directly to leaves, computes utility values, and returns them upward, which is exactly how minimax works.
- BFS does not allow efficient backtracking, does not reach terminal nodes early, and is not compatible with the recursive nature of minimax.

MINMAX Value

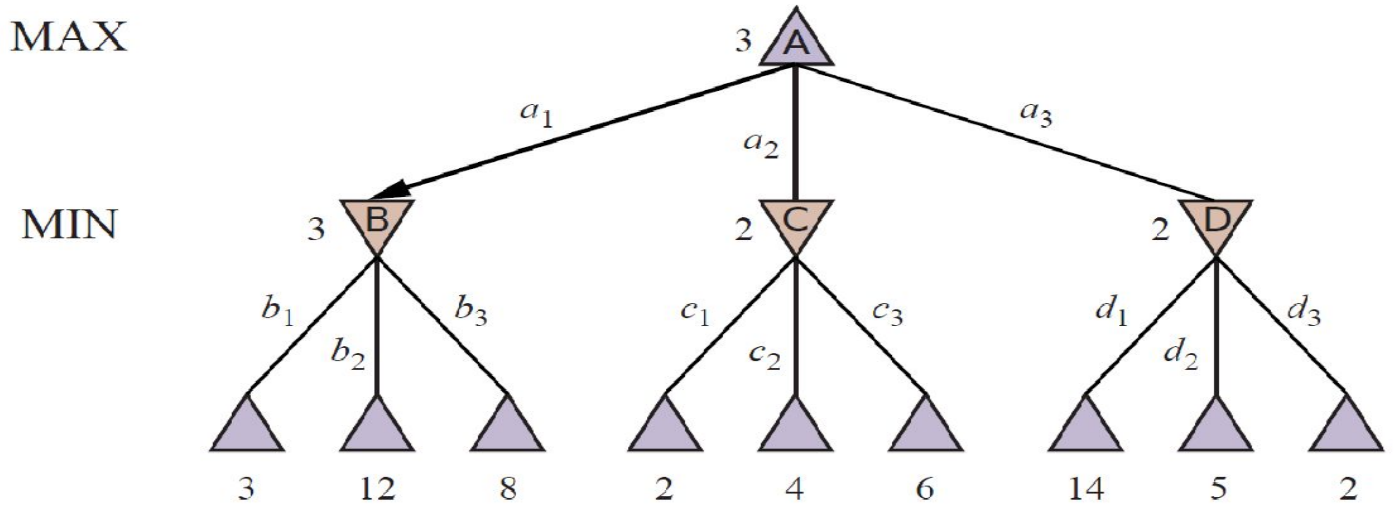
Consider a 'reduced' tic-tac-toe game (because game tree for full tic-tac-toe is too big)

- The possible moves for MAX at the root are a1, a2, and a3
- Possible replies to a1 for MIN are b1, b2, b3, and so on

Optimal strategy can be determined using minimax value of each node MINIMAX(n)

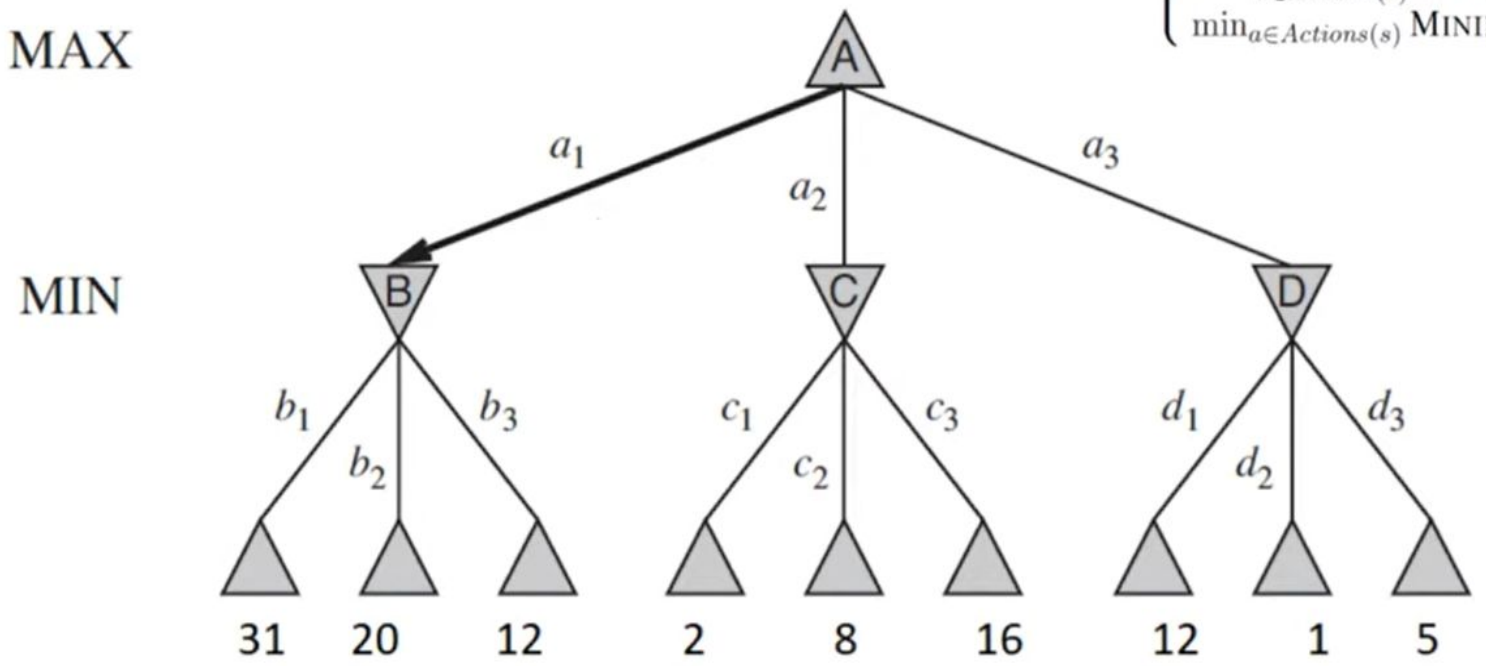
- MINIMAX(n) for the user MAX is the utility of being in the corresponding state
- So, MAX will always prefer to move to a state of maximum value

$$\textsc{Minimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \end{cases}$$

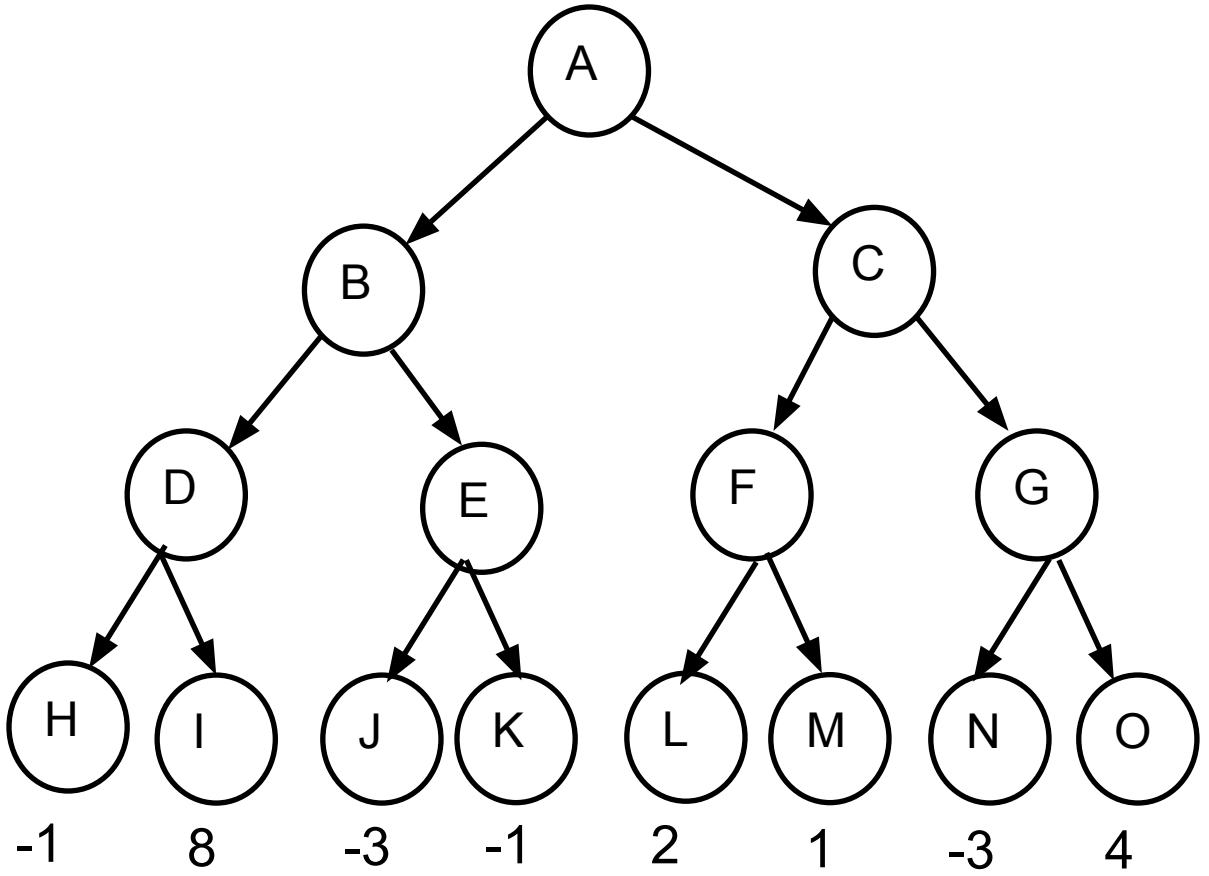


Practice Problem: Calculate the minimax values at nodes A, B, C, and D for the player MAX given the game tree below. The numbers at the leaf nodes represent the values of `Utility (leafnode, MAX)`.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$



Task



Performance Measurement

Complete ?	Optimal ?	Time ?	Space ?
• YES, It will definitely find a solution (if exist), in the finite search tree.	• YES, if both opponents are playing optimally	• The worst case time complexity is $O(b^d)$. Simply $O(n)$	• The worst case space complexity is $O(b^d)$.

Limitation of the minimax Algorithm

- The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc.
- This type of games has a huge branching factor, and the player has lots of choices to decide
  - Traversing becomes complex

- Solution
  - **alpha-beta pruning to reduce the tree and increase efficiency**